



МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ
КАФЕДРА ЕЛЕКТРОННИХ ПРИСТРОЇВ ТА
СИСТЕМ

Лабораторна робота №4
з дисципліни «Організація баз даних»
на тему: «Аналітичні SQL-запити (OLAP)»

Виконав:

ст. гр. ІО-341 Соловійов Руслан

Зараховано від:

Русінов В. В._____

Мета:

Навчитись:

- Використовувати агрегатні функції, такі як COUNT, SUM, AVG, MIN та MAX, для обчислення зведеної статистики з ваших даних.
- Написати запити GROUP BY для групування рядків за одним або кількома стовпцями та обчислення агрегатів для кожної групи.
- Використовувати HAVING для фільтрації результатів згрупованих запитів на основі агрегованих умов.
- Виконувати операції JOIN (принаймні INNER JOIN та LEFT JOIN), щоб об'єднати дані з кількох таблиць.
- Створювати об'єднані запити на агрегацію для кількох таблиць, які об'єднують таблиці та створюють згрупований, агрегований вивід.
- Інтерпретувати результати ваших запитів та пояснити, що робить кожен з них.

Завдання:

Напишіть мінімум 4 запити, що містять агрегаційні функції (SUM, AVG, COUNT, MIN, MAX, GROUP BY), мінімум 3 запити, що використовують різні типи джоїнів (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN), мінімум 3 запити з використанням підзапитів (вибірка з підзапитом у SELECT, WHERE, або HAVING).

Виконайте наступні кроки для завершення лабораторної роботи. Кожен крок описує тип аналітичного запиту, який потрібно написати. Для кожного елемента напишіть SQL-запит у скрипті .sql та запустіть його в pgAdmin для вашої бази даних. Перевірте правильність виводу та переконайтеся, що ви розумієте, що робить запит.

Теоретичні відомості:

Базова агрегація: Напишіть прості запити SELECT, використовуючи агрегатні функції для однієї таблиці.

Приклад - у базі даних студентів знайдіть загальну кількість студентів:

```
SELECT COUNT(*) AS total_students  
FROM Students;
```

Спробуйте використати схожі запити у вашій схемі. Наприклад, підрахуйте загальну кількість замовлень, знайдіть середню оцінку або підсумуйте загальну кількість продажів. Використовуйте COUNT, SUM, AVG, MIN та MAX відповідно.

Групування даних: Використовуйте GROUP BY для групування рядків за одним або кількома стовпцями та обчислення агрегатів для кожної групи.

Приклад - знайдіть кількість студентів за кожною спеціальністю:

```
SELECT major, COUNT(*) AS num_students  
FROM Students  
GROUP BY major;
```

Адаптуйте цей запит до своєї бази даних. Наприклад, групуйте продажі за місяцями, рахуйте студентів за курсами або обчислюйте середню оцінку за кафедрами.

Фільтрування груп: Використовуйте речення HAVING для фільтрації агрегованих результатів.

Приклад - пошук курсів, на яких навчається більше 10 студентів:

```
SELECT course_id, COUNT(*) AS enrolled_count  
FROM Enrollments  
GROUP BY course_id  
HAVING COUNT(*) > 10;
```

Візьміть один із запитів GROUP BY з попереднього кроку та додайте речення HAVING, щоб відфільтрувати групи, які не відповідають умові (наприклад, лише відділи із середнім балом вище 3,0).

Запити JOIN: Об'єднання даних з кількох таблиць за допомогою JOIN.

Приклад (INNER JOIN) - перелічіть ім'я кожного студента з назвами курсів, на яких він навчається:

```
SELECT s.student_name, c.course_title  
FROM Students s
```

JOIN Enrollments e ON s.id = e.student_id

JOIN Courses c ON e.course_id = c.id;

Напишіть у своїй схемі як запит INNER JOIN, так і запит LEFT JOIN.

Наприклад, об'єднуйте таблиці для виведення об'єднаної інформації та використовуйте LEFT JOIN, коли потрібно включити рядки без відповідних записів.

Багатотаблична агрегація: Написання запитів, які об'єднують кілька таблиць та агрегують результати.

Приклад - знайдіть загальну суму, витрачену кожним клієнтом (об'єднання Клієнтів, Замовлень та Елементів Замовлення):

```
SELECT c.customer_name, SUM(oi.quantity * oi.unit_price) AS total_spent
FROM Customers c
JOIN Orders o ON c.id = o.customer_id
JOIN OrderItems oi ON o.id = oi.order_id
GROUP BY c.customer_name;
```

Адаптуйте ці ідеї до своєї бази даних. Використовуйте стільки об'єднань, скільки потрібно, та групуйте за відповідним стовпцем, щоб обчислити потрібний агрегований результат.

Якщо вам потрібні нагадування щодо синтаксису, скористайтеся офіційною документацією PostgreSQL або довідкою pgAdmin. Обов'язково перевірте кожен запит і зрозумійте, чому він видає саме такі результати.

Виконання завдання:

4 команди з агрегаційними функціями:

```
SELECT COUNT(*) AS total_books
FROM book;
```

	total_books	
	bigint	
1		5

```
SELECT status, COUNT(*) AS total_copies
FROM copy
GROUP BY status;
```

	status character varying (20) 🔒	total_copies bigint 🔒
1	on_loan	2
2	available	3
3	damaged	1

```
SELECT
COUNT(*) AS total_loans,
COUNT(*) FILTER (WHERE return_date IS NULL) AS active_loans,
COUNT(*) FILTER (WHERE return_date IS NOT NULL) AS
completed_loans
FROM loan;
```

	total_loans bigint 🔒	active_loans bigint 🔒	completed_loans bigint 🔒
1	4	2	2

```
SELECT
MIN(year_published) AS oldest_book_year,
MAX(year_published) AS newest_book_year,
AVG(year_published)::INT AS avg_year
FROM book;
```

	oldest_book_year integer 🔒	newest_book_year integer 🔒	avg_year integer 🔒
1	1883	1987	1953

3 запити з різними видами JOIN:

```
SELECT * FROM author;
```

	author_id [PK] integer	first_name character varying (100)	last_name character varying (100)	nationality character varying (100)
1	1	Ліна	Костенко	Українська
2	2	Frank	Herbert	Американська
3	3	George	Orwell	Британська
4	4	Іван	Франко	Українська
5	5	Haruki	Murakami	Японська

```
SELECT b.title, a.first_name || ' ' || a.last_name AS author_name
FROM book b
INNER JOIN book_author ba ON ba.book_id = b.book_id
INNER JOIN author a ON a.author_id = ba.author_id
ORDER BY b.title;
```

	title character varying (255)	author_name text
1	1984	George Orwell
2	Dune	Frank Herbert
3	Norwegian Wood	Haruki Muraka...
4	Захар Беркут	Іван Франко
5	Маруся Чурай	Ліна Костенко

```
SELECT b.title, c.copy_id, c.status
FROM book b
LEFT JOIN copy c ON c.book_id = b.book_id
ORDER BY b.title, c.copy_id;
```

	title character varying (255) 🔒	copy_id integer 🔒	status character varying (20) 🔒
1	1984	4	available
2	Dune	3	available
3	Norwegian Wood	6	on_loan
4	Захар Беркут	5	damaged
5	Маруся Чурай	1	available
6	Маруся Чурай	2	on_loan

```

SELECT m.full_name,
       b.title AS borrowed_book,
       l.due_date
FROM member m
LEFT JOIN loan l ON l.member_id = m.member_id AND l.return_date IS
NULL
LEFT JOIN copy c ON c.copy_id = l.copy_id
LEFT JOIN book b ON b.book_id = c.book_id
ORDER BY m.full_name;

```

	full_name character varying (200) 🔒	borrowed_book character varying (255) 🔒	due_date date 🔒
1	Sofía García	[null]	[null]
2	Андрій Коваль	[null]	[null]
3	Василь Петренко	[null]	[null]
4	Марія Іваненко	[null]	[null]
5	Олена Мельник	Маруся Чурай	2025-01-19
6	Тарас Шевченко	Norwegian Wood	2025-01-24

3 запити з використанням підзапитів:

```

SELECT full_name, email
FROM member
WHERE member_id IN (
    SELECT DISTINCT member_id
    FROM loan
    WHERE return_date IS NULL
);

```

	full_name character varying (200) 🔒	email character varying (200) 🔒
1	Олена Мельник	o.melnyk@email.com
2	Тарас Шевченко	t.shevchenko@ukr.net

```

SELECT
    m.full_name,
    (SELECT COUNT(*)
     FROM loan l
     WHERE l.member_id = m.member_id
          AND l.return_date IS NULL) AS active_loans
FROM member m
ORDER BY active_loans DESC;

```

	full_name character varying (200) 🔒	active_loans bigint 🔒
1	Олена Мельник	1
2	Тарас Шевченко	1
3	Марія Іваненко	0
4	Андрій Коваль	0
5	Sofía García	0
6	Василь Петренко	0


```

SELECT ROUND(AVG(loan_count), 2) AS avg_loans_per_member
FROM (
  SELECT member_id, COUNT(*) AS loan_count
  FROM loan
  GROUP BY member_id
) AS member_loan_stats;

```

	avg_loans_per_member numeric
1	1.00

Висновок:

Протягом виконання лабораторної роботи я вивчив агрегатні функції, типи функції JOIN та підзапити, опанувавши ці знання на практиці.